

Certainly. Below are **5 worked examples** for managing a **Linear Queue** and **Circular Queue**, focusing on how `FRONT` , `REAR` , insertion, deletion, and queue contents change after every operation.

1. Linear Queue – Basic Enqueue Operations

Assume:

- Queue size = 5
- Initially: `FRONT = -1` , `REAR = -1`
- `ENQUEUE` adds an element at `REAR` .
- `DEQUEUE` removes an element from `FRONT` .

Operations

`ENQUEUE (10)` , `ENQUEUE (20)` , `ENQUEUE (30)` , `ENQUEUE (40)`

Operation	Queue	FRONT	REAR
Initial	[_, _, _, _, _]	-1	-1
Enqueue 10	[10, _, _, _, _]	0	0
Enqueue 20	[10, 20, _, _, _]	0	1
Enqueue 30	[10, 20, 30, _, _]	0	2
Enqueue 40	[10, 20, 30, 40, _]	0	3

Final:

- `FRONT = 0`
- `REAR = 3`
- Queue = 10 → 20 → 30 → 40

2. Linear Queue – Enqueue and Dequeue

Queue size = 5

Operations:

`ENQUEUE (10)` , `ENQUEUE (20)` , `ENQUEUE (30)` , `DEQUEUE ()` , `DEQUEUE ()` , `ENQUEUE (40)`

Operation	Queue Array	FRONT	REAR	Deleted/Inserted
Initial	[_, _, _, _, _]	-1	-1	—

Operation	Queue Array	FRONT	REAR	Deleted/Inserted
Enqueue 10	[10, _, _, _, _]	0	0	10 inserted
Enqueue 20	[10, 20, _, _, _]	0	1	20 inserted
Enqueue 30	[10, 20, 30, _, _]	0	2	30 inserted
Dequeue	[_ , 20, 30, _, _]	1	2	10 deleted
Dequeue	[_ , _ , 30, _, _]	2	2	20 deleted
Enqueue 40	[_ , _ , 30, 40, _]	2	3	40 inserted

Important point

After two deletions:

```
FRONT = 2
REAR  = 2
```

The queue logically contains:

```
30 → 40
```

The empty spaces at positions **0** and **1** **cannot be reused** by an ordinary linear queue.

3. Linear Queue – Queue Overflow Problem

Queue size = 5

Operations:

```
ENQUEUE(10), ENQUEUE(20), ENQUEUE(30), ENQUEUE(40), ENQUEUE(50), DEQUEUE(),
DEQUEUE(), ENQUEUE(60)
```

Operation	Queue	FRONT	REAR	Status
Initial	[_ , _ , _ , _ , _]	-1	-1	Empty
Enqueue 10	[10, _ , _ , _ , _]	0	0	
Enqueue 20	[10, 20, _ , _ , _]	0	1	

Operation	Queue	FRONT	REAR	Status
Enqueue 30	[10, 20, 30, _, _]	0	2	
Enqueue 40	[10, 20, 30, 40, _]	0	3	
Enqueue 50	[10, 20, 30, 40, 50]	0	4	Full
Dequeue	[_ , 20, 30, 40, 50]	1	4	10 deleted
Dequeue	[_ , _ , 30, 40, 50]	2	4	20 deleted
Enqueue 60	[_ , _ , 30, 40, 50]	2	4	Overflow

Why Overflow?

Although positions **0** and **1** are empty, **REAR = 4**.

For a linear queue:

```
if (REAR == MAX - 1)
    OVERFLOW
```

Therefore, **60** cannot be inserted.

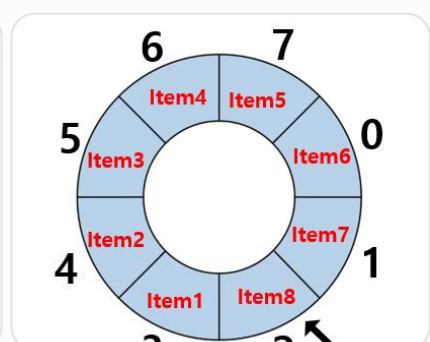
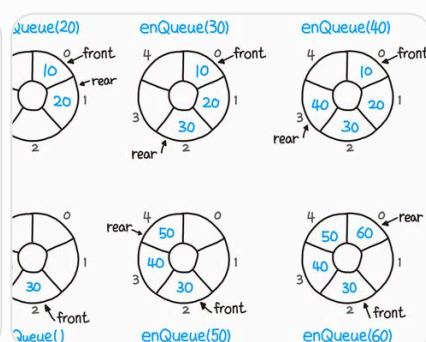
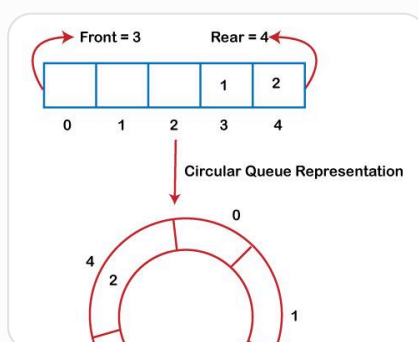
This is one major reason we use a **Circular Queue**.

4. Circular Queue – Reusing Empty Positions

Assume circular queue size = **5**.

Operations:

```
ENQUEUE(10), ENQUEUE(20), ENQUEUE(30), ENQUEUE(40), DEQUEUE(), DEQUEUE(),
ENQUEUE(50), ENQUEUE(60)
```



Operation	Queue Array	FRONT	REAR
Initial	[_,_,_,_,_]	-1	-1
Enqueue 10	[10,_,_,_,_]	0	0
Enqueue 20	[10,20,_,_,_]	0	1
Enqueue 30	[10,20,30,_,_]	0	2
Enqueue 40	[10,20,30,40,_]	0	3
Dequeue	[_,20,30,40,_]	1	3
Dequeue	[_,_,30,40,_]	2	3
Enqueue 50	[_,_,30,40,50]	2	4
Enqueue 60	[60,_,30,40,50]	2	0

Observe the last operation

Before inserting 60:

```
FRONT = 2
REAR  = 4
```

The next position after index 4 is index 0.

Circular movement:

```
0 → 1 → 2 → 3 → 4 → 0 → 1 → ...
```

Therefore:

```
REAR = (REAR + 1) % MAX
      = (4 + 1) % 5
      = 0
```

So 60 is inserted at position 0.

Logical queue is:

```
30 → 40 → 50 → 60
  ↑           ↑
```

5. Circular Queue – Complete Example

Queue size = 5

Perform:

```
ENQUEUE (10)
ENQUEUE (20)
ENQUEUE (30)
ENQUEUE (40)
DEQUEUE ()
DEQUEUE ()
ENQUEUE (50)
ENQUEUE (60)
DEQUEUE ()
ENQUEUE (70)
ENQUEUE (80)
```

Let's calculate every step.

Step	Operation	Array	FRONT	REAR	Logical Queue
0	Initial	[_,_,_,_,_]	-1	-1	Empty
1	Enq 10	[10,_,_,_,_]	0	0	10
2	Enq 20	[10,20,_,_,_]	0	1	10,20
3	Enq 30	[10,20,30,_,_]	0	2	10,20,30
4	Enq 40	[10,20,30,40,_]	0	3	10,20,30,40
5	Deq	[_ ,20,30,40,_]	1	3	20,30,40
6	Deq	[_ ,_,30,40,_]	2	3	30,40
7	Enq 50	[_ ,_,30,40,50]	2	4	30,40,50
8	Enq 60	[60,_,30,40,50]	2	0	30,40,50,60
9	Deq	[60,_,_,40,50]	3	0	40,50,60
10	Enq 70	[60,70,_,40,50]	3	1	40,50,60,70

Step	Operation	Array	FRONT	REAR	Logical Queue
11	Enq 80	[60, 70, 80, 40, 50]	3	2	40, 50, 60, 70, 80

Final situation

Array:

Index	0	1	2	3	4
	60	70	80	40	50
			↑	↑	
			FRONT	REAR	
			3	2	

The array looks unusual because **REAR** has wrapped around.

But the **logical queue order** is:

FRONT

↓

40 → 50 → 60 → 70 → 80

↑

REAR

So even though:

FRONT = 3

REAR = 2

the queue is **full**.

Important Circular Queue Formulas

For an array of size **MAX** :

Enqueue

If queue is not full:

C

```
rear = (rear + 1) % MAX;
```

```
queue[rear] = value;
```

Deque

C

```
value = queue[front];  
front = (front + 1) % MAX;
```

Circular Queue Full Condition

C

```
(rear + 1) % MAX == front
```

Circular Queue Empty Condition

A common implementation uses:

C

```
front == -1
```

Example

If:

```
MAX = 5  
REAR = 4
```

then:

```
(REAR + 1) % MAX  
= (4 + 1) % 5  
= 0
```

Therefore, the next insertion occurs at **index 0**.

Quick Comparison

Feature	Linear Queue	Circular Queue
Movement	One direction	Circular
Rear after last index	Cannot move	Goes to 0

Feature	Linear Queue	Circular Queue
Reuses deleted spaces	✗ No	✓ Yes
Overflow possibility	Wastes space	Better utilization
Full condition	<code>rear == MAX-1</code>	<code>(rear+1)%MAX == front</code>
Main advantage	Simple	Efficient memory use

These examples are particularly useful for students because they show that `FRONT` and `REAR` indicate positions, while the logical queue order may wrap around in a circular queue.